

A principled approach to safety profiles

Document #: P3649R0
Date: 2025-05-19
Project: Programming Language C++
Audience: SG23
Reply-to: Jonathan Müller (think-cell)
<foonathan@jonathanmueller.dev>

Contents

- [1 Abstract](#)
- [2 Background](#)
 - [2.1 Safety](#)
 - [2.2 Profiles](#)
- [3 Proposed design](#)
 - [3.1 Consequence: No false negatives](#)
 - [3.2 Example: Initialization safety](#)
- [4 Concrete proposal](#)
- [5 References](#)

§ 1 Abstract

We should focus on standardizing profiles that give hard guarantees and eliminate entire classes of undefined behavior. We should not use profiles to enforce stylistic rules or add heuristic based checks that make it less likely to encounter undefined behavior. This also requires adding a way to manually mark user-defined functions as violating a profile.

§ 2 Background

§ 2.1 Safety

The term safety in formal methods means a guarantee that a certain invariant holds throughout the lifetime of the program [[Parent2023](#)]. For example, a programming language is memory safe if it guarantees that it is not possible to access invalid memory. The corresponding invariant is “every pointer always points to a valid address”. Safety is useful because it eliminates a whole class of bugs by design.

Note that a safety invariant does not necessarily relate to security vulnerabilities, although it often does. For example, being free from deadlocks is a safety invariant, yet a deadlock is unlikely to be a security vulnerability.

Safety can be achieved by a combination of static analysis (prevent programs that would violate the invariant from compiling) and runtime checks (check and abort if the invariant is violated). It is not possible to achieve 100% safety in system programming languages like C++ or Rust: There are constructs that are safe, yet a static analysis system can't prove it, so has to conservatively reject it, and there are situations where the overhead of runtime checks is not acceptable.

Therefore, Rust is actually two programming languages, “Safe Rust” and “Unsafe Rust”. Safe Rust is a (big) subset of Unsafe Rust, where certain safety invariants are guaranteed. For example, it is impossible to write a Safe Rust program where a reference does not point to a valid object. Unsafe Rust adds a couple more features, such as pointer dereference or union reads, which can violate these invariants. It is the programmers responsibility to ensure that code written in Unsafe Rust does not violate these invariants. Therefore, the transition from Safe Rust to Unsafe Rust requires a big unsafe marker, customary accompanied by a comment explaining why the code does not violate the invariants, and exhaustive testing.

This approach has been proven effective for writing security critical code. The vast majority of Rust code is written in Safe Rust with only small islands of Unsafe Rust in low-level components. Assuming no bugs in the Unsafe Rust code, Safe Rust eliminates a whole class of bugs by design.

§ 2.2 Profiles

[P3589R1] proposes a framework for profiles, a standardized way to enforce certain (compile-time or runtime) checks in C++ code. This can be used to enforce certain clang-tidy checks, MISRA rules, or enable standard library hardening [P3471R4] in a portable way. It can also be used to enforce safety invariants, by banning or checking all constructs that would violate them.

[P3081R2] uses a similar framework to propose a standardized set of profiles. However, those profiles are a mix of heuristics to make it more likely that safety invariants hold (e.g. banning `reinterpret_cast` to avoid type confusion) and stylistic rules (e.g. banning `static_cast` for narrowing conversions). Crucially, the profiles there do not provide a guarantee, a concrete invariant that is being uphold when a given profile is enabled. There is also concern about future evolution of profiles, as there were ideas to extend them with more checks over time. This would make it a breaking change to upgrade to future C++ versions that enforce a profile more strictly.

§ 3 Proposed design

Building on the framework from [P3589R1], I propose that we should initially focus on standardizing safety profiles only. That is, the initial set of standardized profiles should provide a guarantee that some safety invariant holds in their dominion. They should not be used to enforce stylistic rules, modernize old code, or introduce dialects. This goes back to Bjarne’s framework for profiles development [P3274R0], which seem to have gotten lost in [P3081R2].

Concretely, every standardized profile *P* should have an associated invariant. Enabling *P* guarantees that this invariant holds for the entire dominion of profile *P*. This is done in a combination of static analysis rejections (e.g. disallowing pointer arithmetic) and/or runtime checks (e.g. checking array access). The only way to violate the invariant associated with *P* is in code outside the dominion of *P*, e.g.:

- Code explicitly suppressing the profile within the translation unit that would normally be in the dominion of *P* using the `[[profiles::suppress(P)]]` attribute.
- Code written in dependencies that have not (yet) enabled *P* (e.g. a third-party or C library).
- Code injected by users that have not enabled *P* (e.g. a callback passed to the dominion of *P*).

That way, we have achieved an “island of safety” in our C++ code, which can be over time expanded to a “continent of safety” by enabling profiles in more and more code. Furthermore, when using `[[profiles::require(P)]]` on all import statements, which ensures that code in dependencies also enable *P*, we achieve the Rust model, where we have a safety guarantee in all but a few, explicitly marked with `[[profiles::suppress(P)]]`, sections of unsafe code. As Rust has shown, this approach works.

This principled approach to profile selection, both avoids endless debates about stylistic issues, as there is now a clear technical reason to why a particular construct is banned by a profile (it would violate the invariant in a way that cannot be checked), and further ensures that profiles are stable and backwards compatible (if we do our jobs right, we won’t later discover more rules that would apply to existing code). This side steps a lot of the discussion around the profiles in [P3081R2].

§ 3.1 Consequence: No false negatives

To guarantee safety, the invariant needs to be enforced, not just mostly enforced. 100% of code that violates the invariant needs to be rejected or checked at runtime.

Otherwise, it is not a safety invariant, but more a safety heuristic.

This safety heuristic approach is what C++ has already been doing for 40 years: Smart pointers are a heuristic approach to prevent use-after-free, constructors are a heuristic approach to prevent uninitialized memory, and so on.

And this approach is precisely why C++ is facing regulatory pressure: Heuristics do not provide guarantees that can be relied upon. If anything, heuristics lull you into a false sense of security, as it is a lot less likely that you run into bugs, so you start being less careful.

If you believe that there is serious regulatory pressure to make C++ safe, do you think they will be satisfied if we keep doing what we have been doing for 40 years? We need a paradigm shift, not a continuation of the status quo.

Furthermore, why would someone go through the trouble of enabling a profile, updating a lot of code to follow its rules, without actually getting a concrete guarantee out of it?

§ 3.2 Example: Initialization safety

One simple example of a safety profile is an initialization profile. The associated safety invariant is “every variable is initialized before it is used”.

One easy way to enforce this invariant is to disallow default initialization of variables, unless it is a class type with a non-trivial default constructor, and constructs such as `reinterpret_cast` which would allow the creation of a pointer without calling any constructor. Furthermore, it should also prohibit calling standard library functions that return uninitialized memory, like `std::malloc`. By exhaustively enumerating and prohibiting all ways to create objects without initializing them, we can guarantee that the invariant holds.

This static analysis is very simple, but has obvious false positives, such as:

```
int x; // error: x is not initialized
x = 5;
use(x);
```

The code itself is correct and does not violate the safety invariant, yet our static analysis would reject it. A more sophisticated static analysis could do flow analysis and determine that `x` is initialized before use. When standardizing this hypothetical profile, we need to decide whether we should require a specific static analysis scheme, and if so, which one, or whether we allow an implementation to do more sophisticated static analysis which would lead to fewer false positives.

This profile also highlights a potential hole in the framework. Let’s say that in order to ensure the initialization safety invariant, we have decided to ban `std::malloc` as part of the enforcement. What about a third-party function that defines `my_malloc`? It also needs to be banned, yet the profile has no way of knowing that.

Therefore, we need an attribute to indicate that something is an unsafe abstraction: It builds on top of constructs prohibited by the profile in such a way that it could potentially violate the invariant. Let’s use the attribute syntax `[[profiles::prohibit_in(P)]]` to indicate that something also needs to be banned when `P` is used:

```
[[profiles::prohibit_in(initialization)]] void* my_malloc(std::size_t);
```

Note the distinction between `[[profiles::suppress(P)]]` and `[[profiles::prohibit_in(P)]]`: `[[profiles::suppress(P)]]` means “this construct is banned by P, because we cannot prove that the invariant holds, but in this particular context, I know it holds”, whereas `[[profiles::prohibit_in(P)]]` means “this construct also needs to be banned by P because it could potentially violate the invariant”. Note that by definition, the implementation of a `[[profiles::prohibit_in(P)]]` function has to use `[[profiles::suppress(P)]]` at some point (or third-party code that is not in the dominion of P): After all, if P is enabled, the invariant cannot be violated! However, not every use of `[[profiles::suppress(P)]]` needs to be marked with `[[profiles::prohibit_in(P)]]`: If you do something with the suppressed construct that would not violate the invariant, the suppression does not need to propagate.

For example, if we want to use `std::malloc` to allocate memory and initialize it, we don’t need to mark the composed construct with `[[profiles::prohibit_in(initialization)]]`:

```
void* zeroed_malloc(std::size_t size)
{
    [[profiles::suppress(initialized)]] auto ptr = std::malloc(size);
    std::memset(ptr, 0, size);
    return ptr;
}
```

However, if we want to provide a convenient wrapper for allocating an object with `std::malloc` without initialization, we do need to mark the composed construct:

```
template <typename T>
[[profiles::prohibit_in(initialization)]] T* allocate()
{
    [[profiles::suppress(initialization)]] return static_cast<T*>
        (std::malloc(sizeof(T)));
}
```

(In Rust, both `[[profiles::suppress(P)]]` and `[[profiles::prohibit_in(P)]]` are spelled with the `unsafe` keyword.)

Another consequence is that a construct might need to be banned for multiple profiles. For example, `reinterpret_cast` can both lead to accessing uninitialized objects, as well as strict aliasing violations. It therefore needs to be suppressed in both a hypothetical initialization profile and a `type_confusion` profile, and the framework needs to allow for that, by e.g. allowing a variadic `[[profiles::suppress(P, Q, ...)]]` or a blanket `[[profiles::suppress]]` syntax.

§ 4 Concrete proposal

WG21 should focus on standardizing safety profiles only (at least at first). Each safety profiles gives a guarantee that a certain safety invariant, usually a guarantee that a certain

set of undefined behavior does not happen, holds in its dominion. This is hard guarantee, not a heuristic; false negatives are not acceptable.

Every proposed profile should make it clear what invariant it enforces, and how it is enforced; similar to the approach in [P3274R0].

Furthermore, [P3589R1] should be extended with a `[[profiles::prohibit_in(P)]]` attribute (or syntax like it) to indicate constructs that should be banned when profile P is enabled, as well as a way to suppress a construct in multiple profiles at once.

§ 5 References

[P3081R2] Herb Sutter. 2025-02-03. Core safety profiles for C++26.

<https://wg21.link/p3081r2>

[P3274R0] Bjarne Stroustrup. 2024-05-10. A framework for Profiles development.

<https://wg21.link/p3274r0>

[P3471R4] Konstantin Varlamov, Louis Dionne. 2025-02-14. Standard Library Hardening.

<https://wg21.link/p3471r4>

[P3589R1] Gabriel Dos Reis. 2025-02-03. C++ Profiles: The Framework.

<https://wg21.link/p3589r1>

[Parent2023] Keynote: Safety in C++: All the Safeties! - Sean Parent - C++ on Sea 2023.

<https://www.youtube.com/watch?v=BaUv9sgLCPc>